

Transactions

Rigorous-2PL Rules Summary

Rigorous 2PL ensures **serializability** by enforcing strict lock acquisition and release policies:

Lock Types:

- **Shared (S) Lock:** for reading (R).
- **Exclusive (X) Lock:** for writing (W).
 - Only one transaction can hold an **X lock**.
 - **S locks** can be shared **unless** an **X lock** is already held or requested.

Locking Behavior:

- Before a transaction **reads**, it must obtain a **shared (S)** lock.
- Before a transaction **writes**, it must obtain an **exclusive (X)** lock.
- **All locks are held until COMMIT/ABORT.**
 - No lock is released earlier (as in strict or basic 2PL).
 - This avoids cascading rollbacks.

Deadlocks:

- **Deadlock occurs** when a **cycle** exists in the "waits-for" graph.
 - T1 waits for T2, T2 waits for T3, ..., Tn waits for T1.

How to Detect and Understand Deadlocks (via your example)

Let's consider an example from the table:

Timestep	T1	T2	T3	T4
1	<i>BEGIN</i>			
2		<i>BEGIN</i>		
3	W(C)			
4		R(A)		
5			<i>BEGIN</i>	
6	R(B)			
7				<i>BEGIN</i>
8		W(B)		
9			W(C)	
10		<i>COMMIT</i>		
11				R(A)
12			<i>COMMIT</i>	
13				<i>COMMIT</i>
14	W(A)			
15	<i>COMMIT</i>			

Correct Analysis:

- **T1 wants to Read(B) at step 6, but:**
 - **T2 writes B at step 8** → needs X lock on B.
- If **T1 has a shared lock** on B at step 6 and holds it (R2PL), then T2 **must wait**.
- Conversely, if **T2 locks B** first (step 8), then **T1 will wait** for shared lock.

This causes **mutual waiting** ⇒ **Deadlock** possible between T1 and T2.

So this statement is true:

Transaction 1 and 2 result in a deadlock

Key Takeaways for Exam:

When is there NO Deadlock?

If two transactions **never request the same lock**, or one finishes (commits) before the other requests the same resource, no deadlock will occur.

Example:

If T1's R(B) happens at step 8 and T2's W(B) at step 6 — order matters, so no deadlock.

Summary of Your Multiple-Choice Answers

Statement	Correct?	Why
If T1's R(B) at step 8 and T2's W(B) at step 6, no deadlock	true	Lock order avoids mutual wait
T1 and T2 result in deadlock	true	Both want B, and hold until commit
Only T3 and T4 run to completion	false	T3 reach a deadlock
T4 finishes first	true	Only T4 commits
If T2 is aborted, T1 and T3 deadlock	false	T1 would still run to completion

Let me know if you'd like to practice identifying waits-for graphs or simulate more schedules!